



東北大學

学院 资源与土木工程学院

班级 智能采矿工程 2302

教师 李风云

学号 20233049

姓名 袁修

2025 年 10 月 20 日

目录

1	引言	3
1.1	项目背景	3
1.2	设计目标	3
2	可行性分析	3
2.1	技术可行性	3
2.2	经济可行性	3
2.3	运行可行性	4
3	数据库需求分析	4
3.1	功能需求	4
3.1.1	用户管理功能	4
3.1.2	商品管理功能	4
3.1.3	订单处理功能	4
3.1.4	地址管理功能	4
3.1.5	商品分类功能	5
3.1.6	功能结构图	5
3.2	性能需求	5
3.2.1	数据查询性能	5
3.2.2	数据操纵性能	5
3.2.3	数据准确性性能	6
4	数据库结构设计	6
4.1	概念结构设计 (E-R 图)	6
4.1.1	设计局部 E-R 模型	6
4.1.2	设计全局 E-R 模型	8
4.1.3	优化全局 E-R 模型	9
4.2	逻辑结构设计 (MySQL 表结构)	10
4.3	物理结构设计	14
5	数据库行为设计	14
5.1	数据查询行为	14
5.1.1	单表查询	14
5.1.2	连接查询	14
5.1.3	嵌套查询	15
5.2	数据操纵行为	15
5.2.1	插入数据	15

5.2.2	修改数据	15
5.2.3	删除数据	15
5.3	视图操作行为	15
5.3.1	用户订单概览视图	15
5.3.2	商品销售统计视图	16
5.3.3	订单商品明细视图	16
5.3.4	用户地址关联视图	16
5.3.5	商品分类关联视图	16
6	数据库实施	16
6.1	数据库创建与表定义	16
6.2	插入虚拟数据	20
6.3	数据库行为实际操作	23
6.3.1	数据查询	23
6.3.2	数据操作	28
6.3.3	视图操作	31

1 引言

1.1 项目背景

在互联网购物持续普及的趋势下，“学生日常购物”场景对简易电商系统的需求日益凸显，为深入理解数据库设计完整流程、掌握 MySQL 实际应用，本项目以支撑用户注册、商品浏览、下单支付等基础电商功能为核心，开展电商系统数据库设计工作。

1.2 设计目标

1. 严格遵循“需求分析 - 结构设计 - 行为设计 - 实施”的标准流程，完成电商系统数据库的全流程设计。
2. 设计简洁、合理的表结构，确保数据完整性和关联性。
3. 基于 MySQL 实现数据库创建、虚拟数据插入及数据查询、操纵、控制等基础操作。
4. 模拟用户注册登录、商品浏览、订单处理等电商核心业务场景，验证数据库的可用性。

2 可行性分析

2.1 技术可行性

本电商系统数据库设计采用 MySQL 作为核心数据库，其功能特性能够充分满足系统需求。MySQL 支持 InnoDB 存储引擎，可通过外键约束保障用户、商品、订单等数据间的关联完整性；提供 DECIMAL 类型用于精确存储商品价格等数值型数据，通过 CHECK 约束实现库存非负等业务规则，同时支持索引功能以优化查询效率。开发工具选用 MySQL Workbench，该工具提供可视化的表结构设计界面、SQL 语句编辑器及查询结果验证功能，能够直观高效地完成数据库建模、脚本编写与功能测试等工作。

2.2 经济可行性

本项目在资源投入方面具有显著的经济性。硬件层面，仅需普通个人计算机即可满足开发与测试需求，无需专用服务器设备；软件层面，MySQL 数据库及 MySQL Workbench 工具均为开源免费产品，无版权费用支出。人力成本方面，基于明确的需求边界与成熟的技术方案，1 名开发者可在 1-2 周内完成从设计到实施的全流程工作。项目成果能够有效转化为数据库设计与 SQL 操作的实践经验，为后续系统开发奠定基础，投入产出比合理，符合实践项目的资源配置特点。

2.3 运行可行性

本数据库系统能够稳定支撑中小型电商场景的运行需求。针对学生购物这一核心场景，系统设计的用户表、商品表、订单表等结构可高效存储与处理数千级用户数据、数百级商品数据及数万级订单数据，在常规并发访问下能保持良好性能。用户注册登录、商品浏览下单等核心业务流程对应的 SQL 操作逻辑清晰，通过基础数据操纵语句即可实现；管理员可借助 MySQL Workbench 便捷地完成商品信息维护、订单状态更新等日常操作，整体运行流程简洁可控，操作门槛符合实际使用需求。

3 数据库需求分析

3.1 功能需求

3.1.1 用户管理功能

需支持用户注册与登录的数据交互，存储用户唯一标识 (user_id)、用户名 (name)、手机号 (phone)、登录密码 (password) 及注册时间 (reg_time)，其中手机号需满足唯一性约束（用于登录验证，避免重复注册），注册时间可自动生成（默认取当前时间），确保用户数据的完整性与唯一性。

3.1.2 商品管理功能

需实现商品信息的存储与查询，存储商品唯一标识 (product_id)、商品名称 (pname)、商品价格 (price) 及库存数量 (stock)，价格需保留 2 位小数（精准反映商品价值），库存需满足非负约束（防止超卖问题）；同时支持商品列表查询（如按价格筛选低价商品）与库存更新（下单后扣减对应商品库存），确保商品数据实时准确。

3.1.3 订单处理功能

需覆盖订单创建、订单项关联及订单查询全流程，存储订单唯一标识 (order_id)、关联用户 ID (user_id)、下单时间 (order_time) 及订单总金额 (total)，总金额需满足非负约束且等于订单项金额之和；订单项需存储订单项唯一标识 (item_id)、关联订单 ID (order_id)、关联商品 ID (product_id)、购买数量 (num) 及下单时单价 (price)，购买数量需满足 1 约束，下单时单价需固定存储（避免商品调价影响历史订单金额），同时支持订单与订单项的关联查询（如查看订单对应的商品明细），确保订单数据的关联性与可追溯性。

3.1.4 地址管理功能

需支持用户对收货地址的新增与编辑操作，可设置默认地址，存储地址唯一标识 (addr_id)、关联用户 ID (user_id)、收货人姓名 (receiver)、收货人电话 (phone) 及

详细地址 (detail) 等信息，以保障用户下单时能快速选取已存储的地址（与地址表相关联），提升订单填写效率。

3.1.5 商品分类功能

需支持用户依据商品分类来筛选商品，例如从“电脑设备”分类中进一步筛选“电脑配件”类商品，同时管理员能够对分类信息进行维护，存储分类唯一标识 (category_id)、分类名称 (category_name) 以及父分类关联 (parent_id) 等内容（与商品分类表相关联），便于商品的分类管理与用户的精准查找。

3.1.6 功能结构图

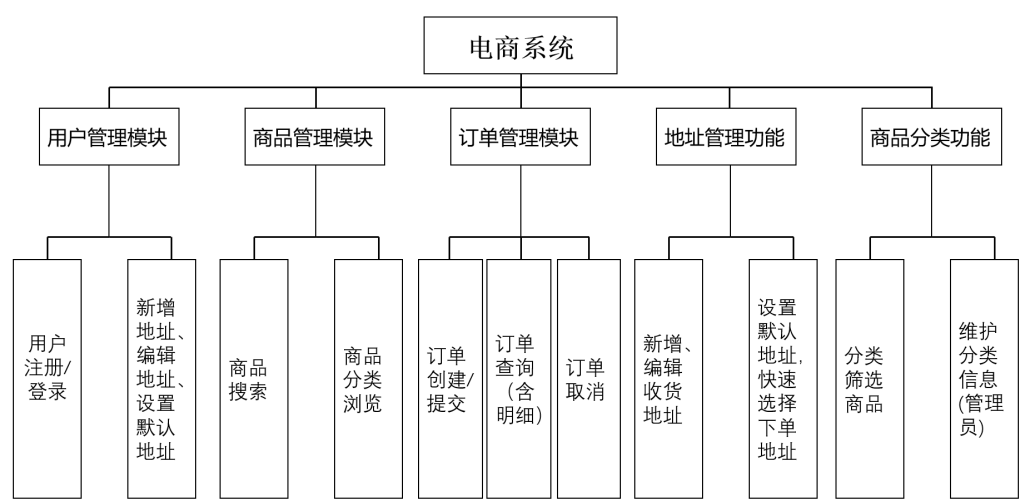


图 1: 功能结构图

3.2 性能需求

3.2.1 数据查询性能

针对高频查询场景需保障响应效率，如用户登录时查询手机号与密码（需通过索引优化，为 user 表 phone 字段建立唯一索引）、用户查询个人订单列表（为 order 表 user_id 建立普通索引）、管理员查询商品销售记录（为 order_item 表 product_id 建立普通索引），确保单次查询响应时间控制在合理范围，避免用户操作等待过长。

3.2.2 数据操纵性能

支持批量数据插入（本文初始化 8 条用户数据、8 条商品数据）与并发数据更新（如多用户同时下单时的库存扣减、订单创建），需通过事务控制（确保“查询库存 - 扣减库存 - 生成订单 - 关联订单项”操作的原子性）避免数据不一致，同时在常规并发访问（如数十用户同时浏览商品、下单）下，保障数据库操作无延迟、无数据丢失。

3.2.3 数据准确性性能

需通过约束机制确保数据完整性，主键约束（如 user_id、product_id 等唯一标识非空且唯一）保障数据唯一性，外键约束（如 order 表 user_id 关联 user 表 user_id、order_item 表 order_id 关联 order 表 order_id）保障数据关联性，域约束（如 price 0、stock 0、num 1）保障数据有效性，避免因数据错误导致业务异常（如超卖、订单金额为负）。

4 数据库结构设计

4.1 概念结构设计 (E-R 图)

4.1.1 设计局部 E-R 模型

1. **用户**：属性为 user_id（主键）、name（姓名）、phone（手机号）、password（密码）、reg_time（注册时间）

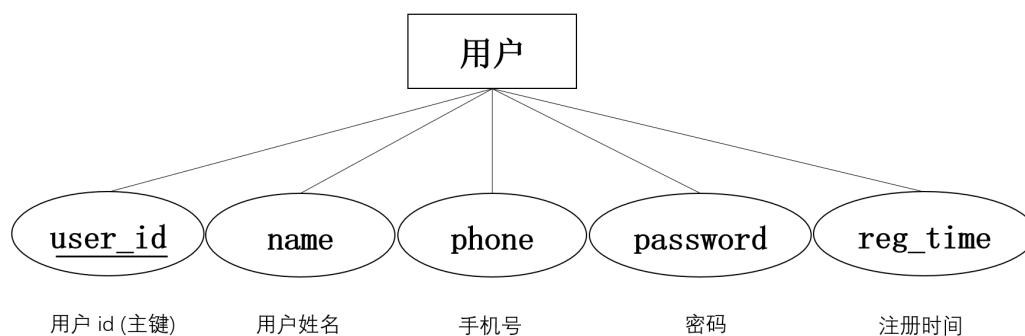


图 2: 用户 E-R 图

2. **商品**：属性为 product_id（主键）、pname（商品名称）、price（价格）、stock（库存）、category_id（分类 id，外键）

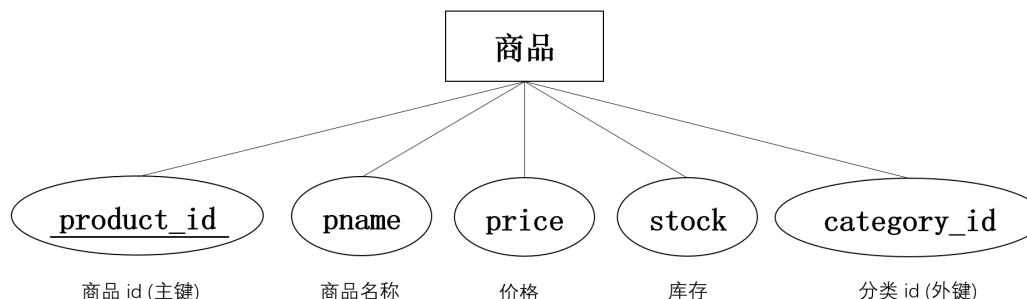


图 3: 商品 E-R 图

3. **订单**：属性为 order_id（主键）、user_id（外键）、order_time（下单时间）、total（总金额）

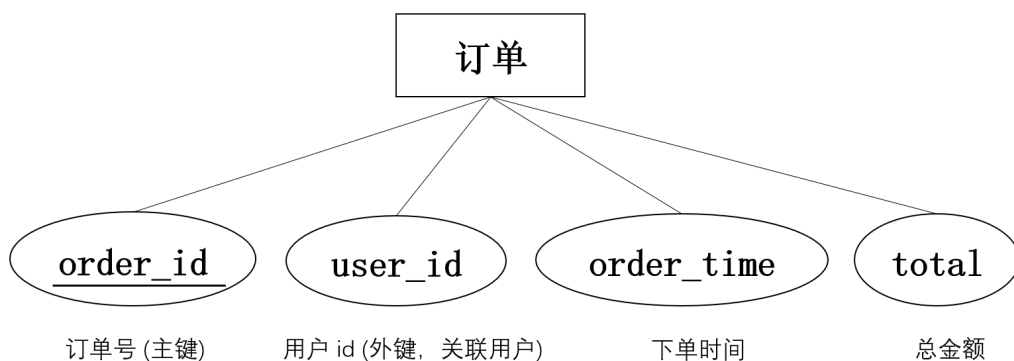


图 4: 订单 E-R 图

4. **订单项**: 属性为 item_id (主键)、order_id (外键)、product_id (外键)、num (购买数量)、price (购买单价)

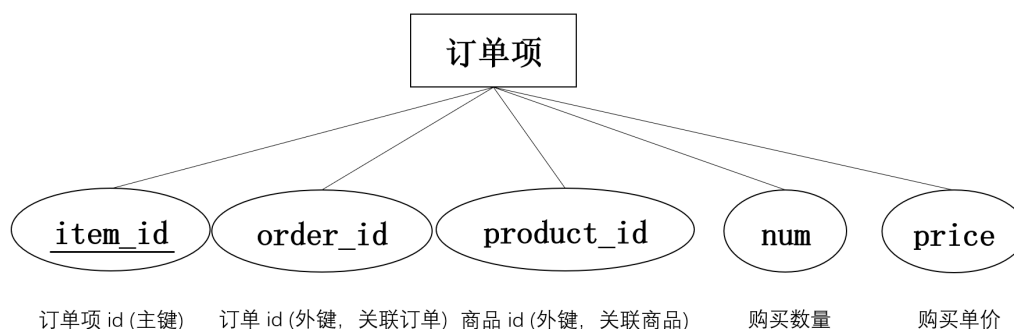


图 5: 订单项 E-R 图

5. **地址**: 属性: addr_id (主键)、user_id (外键)、receiver (收件人姓名)、phone (收件人电话)、detail (详细地址)、is_default (是否设为默认收货地址)

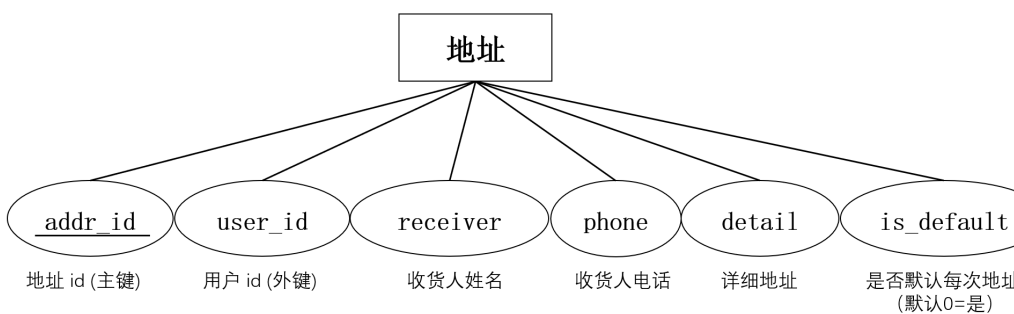


图 6: 地址 E-R 图

6. **商品分类**: 属性: category_id (主键)、category_name (分类名称)、parent_id (外键, 自关联)

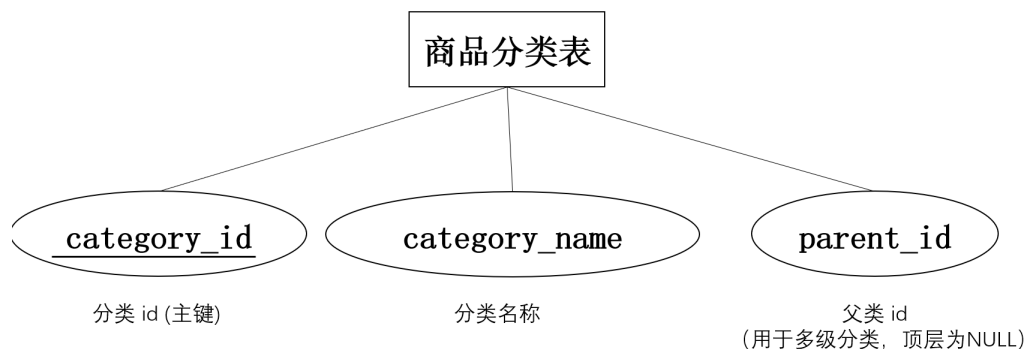


图 7: 商品分类 E-R 图

4.1.2 设计全局 E-R 模型

建立实体间的关系：

1. “用户”与“商品”：多对多关系（多个用户可浏览多种商品，一种商品可被多个用户浏览）。
2. “用户”与“订单”：一对多关系（1 个用户可下多个订单，1 个订单属于 1 个用户）。
3. “订单”与“商品”：多对多关系（一个订单可包含多种商品，一种商品可出现在多个订单中）。
4. “订单”与“订单项”：一对多关系（1 个订单包含多个订单项，1 个订单项属于 1 个订单）。
5. “商品”与“订单项”：一对多关系（1 种商品可出现在多个订单项，1 个订单项对应 1 种商品）。
6. “商品分类”与“商品”：一对多关系（1 种商品分类包含多个商品，1 种商品属于 1 个商品分类）。
7. “用户”与“地址”：一对多关系（1 个用户有多个地址，1 个地址属于 1 个用户）。

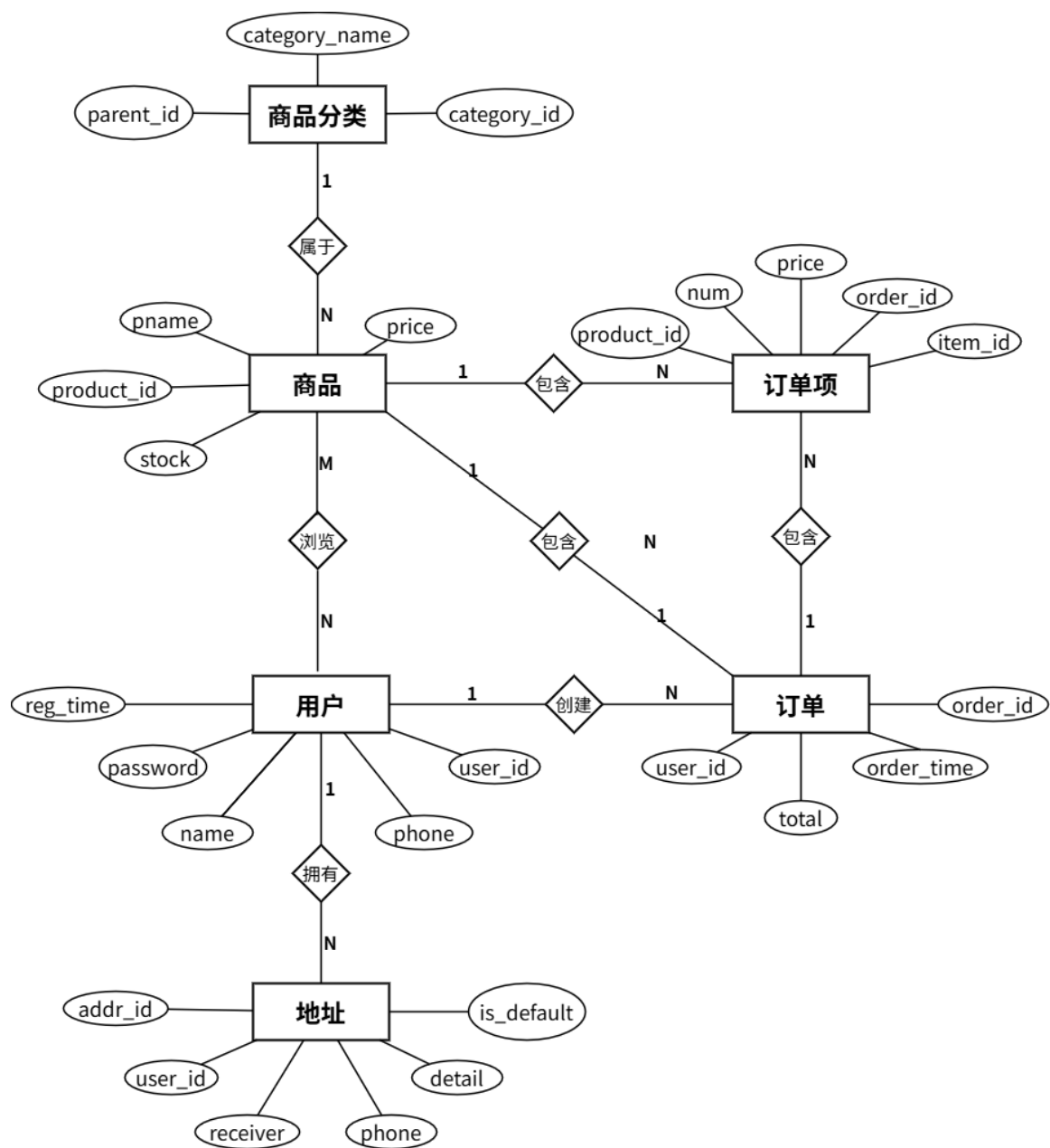


图 8: 全局 E-R 图

4.1.3 优化全局 E-R 模型

为了聚焦核心业务逻辑“用户下单 - 订单包含订单项 - 订单项关联商品”，消除冗余关系，让模型更简洁清晰，便于后续数据库设计与理解。在全局 E - R 模型简化时，去除“用户 - 商品”的多对多“浏览 / 关注”关系（若核心业务为下单购买，此关系非核心且可通过订单间接关联）以及“订单 - 商品”的多对多“选购”关系（订单项已能更精准体现订单与商品的关联并包含购买细节）。

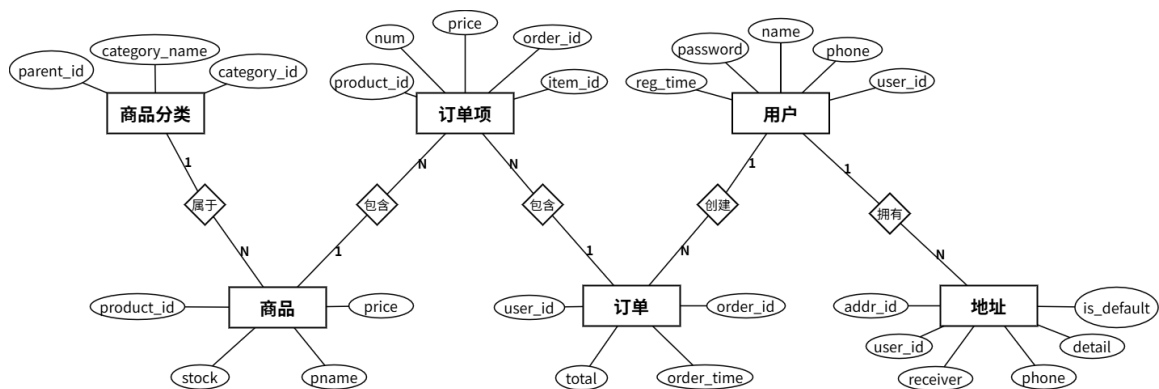


图 9: 全局 E-R 图 (优化后)

4.2 逻辑结构设计 (MySQL 表结构)

将 E-R 图转换为 4 张 MySQL 表，结构如下：

1. 用户表 (user)

字段名	数据类型	约束	说明
user_id	INT	PRIMARY KEY	用户唯一标识
name	VARCHAR (10)	NOT NULL	用户名
phone	VARCHAR (20)	NOT NULL, UNIQUE	手机号 (唯一)
password	VARCHAR (20)	NOT NULL	登录密码
reg_time	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	注册时间 (自动生成)

表 1: 用户表结构

2. 商品表 (product)

字段名	数据类型	约束	说明
product_id	INT	PRIMARY KEY	商品唯一标识
pname	VARCHAR (50)	NOT NULL	商品名称
price	DECIMAL (10,2)	NOT NULL, CHECK (price >= 0)	商品价格 (保留 2 位小数, 非负)
stock	INT	NOT NULL, DEFAULT 0, CHECK (stock >= 0)	商品库存 (默认 0, 非负)
索引		INDEX idx_product_name (pname)	普通索引 (搜索商品)

表 2: 商品表结构

3. 订单表 (order)

字段名	数据类型	约束	说明
order_id	INT	PRIMARY KEY	订单唯一标识
user_id	INT	NOT NULL	关联用户表
order_time	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	下单时间 (自动生成)
total	DECIMAL (10,2)	NOT NULL, CHECK (total >= 0)	订单总金额 (非负)
外键约束		FOREIGN KEY (user_id) REFERENCES ecom- merce.user(user_id)	确保订单关联存在的 用户
索引		INDEX idx_order_user (user_id)	普通索引 (查用户的所有订单)

表 3: 订单表结构

4. 订单项表 (order_item)

字段名	数据类型	约束	说明
item_id	INT	PRIMARY KEY	订单项唯一标识
order_id	INT	NOT NULL	关联订单表
product_id	INT	NOT NULL	关联商品表
num	INT	NOT NULL, CHECK (num > 0)	购买数量（大于等于 1）
price	DECIMAL (10,2)	NOT NULL, CHECK (price >= 0)	下单时单价（非负）
外键约束		FOREIGN KEY (order_id) REFERENCES ecommerce.order(order_id)	确保订单项关联存在的订单
外键约束		FOREIGN KEY (product_id) REFERENCES ecommerce.product(product_id)	确保订单项关联存在的商品
索引		INDEX idx_item_order (order_id)	普通索引（查订单详情）
索引		INDEX idx_item_product (product_id)	普通索引（查商品销售记录）

表 4: 订单项表结构

5. 地址表 (address)

字段名	数据类型	约束	说明
addr_id	INT	PRIMARY KEY	地址唯一标识
user_id	INT	NOT NULL	关联用户表（外键关联 user 表 user_id）
receiver	VARCHAR(20)	NOT NULL	收货人姓名
phone	VARCHAR(20)	NOT NULL	收货人电话（支持与用户注册电话不同）
detail	VARCHAR(100)	NOT NULL	详细地址（如街道、门牌号、小区名称）
is_default	TINYINT	NOT NULL, DEFAULT 0	是否默认地址 (0= 否, 1= 是)
外键约束		FOREIGN KEY (user_id) REFERENCES ecom-merce.'user'(user_id)	确保地址关联存在的用户
索引		INDEX idx_addr_user (user_id)	普通索引（优化用户地址查询效率）

表 5: 地址表结构

6. 商品分类表 (category)

字段名	数据类型	约束	说明
category_id	INT	PRIMARY KEY	分类唯一标识
category_name	VARCHAR(30)	NOT NULL, UNIQUE	分类名称（如“电脑设备”“输入设备”）
parent_id	INT	DEFAULT NULL	父分类 ID（顶级分类为 NULL, 支持多级分类）
外键约束		FOREIGN KEY (parent_id) REFERENCES ecom-merce.category(category_id) ON DELETE SET NULL	自关联约束，父分类删除时子分类 parent_id 设为 NULL
补充说明		-	支持多级分类逻辑（如“电脑设备 → 电脑配件 → 输入设备”）

表 6: 商品分类表结构

4.3 物理结构设计

1. **存储引擎**：所有表使用 InnoDB 引擎，支持外键约束，确保数据关联完整性
2. **字符集**：使用 utf8mb4 字符集，支持中文，表情符号，特殊符号等
3. **索引**：仅为高频查询字段添加简单索引

(1) 商品表 (product)：idx_product_name，是普通索引，用于搜索商品，提升根据商品名称查询商品的效率。

(2) 订单表 (order)：idx_order_user，为普通索引，用于查看用户的所有订单，提高通过用户 ID 查询订单的速度。

(3) 订单项表 (order_item)：idx_item_order，是普通索引，用于查看订单详情，加快通过订单 ID 查询订单项的效率；idx_item_product，为普通索引，用于查看商品销售记录，提升通过商品 ID 查询订单项的速度。

(4) 地址表 (address)：idx_addr_user，是普通索引，用于优化用户地址查询效率，方便通过用户 ID 查询地址信息。

5 数据库行为设计

5.1 数据查询行为

5.1.1 单表查询

- 基础查询：查询指定表的全部或部分字段数据
- 条件查询：带 WHERE 子句的过滤查询
- 排序查询：使用 ORDER BY 对结果排序
- 聚合查询：使用 COUNT、SUM、AVG、MAX、MIN 等聚合函数
- 分组查询：使用 GROUP BY 进行数据分组及 HAVING 筛选

5.1.2 连接查询

- 等值连接：基于相等条件的多表连接
- 非等值连接：基于非相等条件的多表连接
- 自连接：同一张表内的连接查询
- 外连接：保留未匹配的行（左外连接、右外连接）

5.1.3 嵌套查询

- 子查询作为条件：返回单值或多值的子查询
- 子查询作为数据源：FROM 子句中的子查询
- 相关子查询：依赖外部查询参数的子查询

5.2 数据操纵行为

5.2.1 插入数据

- 插入单行数据
- 插入多行数据
- 从查询结果插入数据

5.2.2 修改数据

- 修改表中指定行的字段值
- 基于条件的批量更新
- 关联多表的更新操作

5.2.3 删除数据

- 删除指定条件的记录
- 清空表数据
- 关联多表的删除操作

5.3 视图操作行为

5.3.1 用户订单概览视图

该视图通过关联用户表和订单表实现，选取用户 ID、用户名、订单 ID、下单时间及订单总金额等核心字段。实现时需建立用户表与订单表的左连接，以确保完整呈现所有用户的订单情况，包括暂无订单的用户。此视图的作用在于方便快速查询特定用户的订单历史及消费金额，简化多表关联的查询操作，提升用户订单管理效率。

5.3.2 商品销售统计视图

此视图基于商品表和订单项表构建，包含商品 ID、商品名称、销售数量及销售总额等关键信息。实现过程中需对订单项表按商品 ID 进行分组汇总，计算每个商品的累计销售数量和总金额，并与商品表关联获取商品基本信息。其作用是直观展示各商品的销售业绩，为管理员调整库存、制定促销策略提供数据支持，减少重复统计计算的工作量。

5.3.3 订单商品明细视图

该视图整合订单表、订单项表和商品表的信息，涵盖订单 ID、下单时间、商品名称、购买数量、商品单价及单项金额等内容。实现时需通过订单 ID 关联订单表与订单项表，再通过商品 ID 关联订单项表与商品表，形成完整的订单商品链条。此视图的作用是清晰呈现每笔订单所包含的商品详情，便于用户核对订单信息和管理员处理订单售后，增强订单数据的可读性和实用性。

5.3.4 用户地址关联视图

该视图关联用户表与地址表，涵盖用户 ID、用户名、收货人、收货电话及详细地址等信息。通过左连接用户表与地址表，完整呈现用户的收货地址情况，包括默认地址，便于快速查询用户的地址信息，为下单选地址等操作提供数据支持，简化多表关联查询步骤。

5.3.5 商品分类关联视图

此视图基于商品表和分类表构建，包含商品 ID、商品名称、价格以及所属分类名称等信息。实现时通过商品 ID 关联商品表与分类表，能直观展示商品所属分类，方便用户按分类筛选商品，也为管理员管理商品分类提供数据参考，减少分类相关查询的操作复杂度。

6 数据库实施

6.1 数据库创建与表定义

执行以下 MySQL 代码，创建数据库和 4 张核心表：

```
-- 创建数据库
CREATE DATABASE ecommerce;

-- 1. 用户表
CREATE TABLE ecommerce.`user` -- 关键字加反引号避免冲突
(
```

```

user_id INT PRIMARY KEY,
`name` VARCHAR(10) NOT NULL,
phone VARCHAR(20) NOT NULL UNIQUE, -- 每个手机号只能注册一个账号，唯一索引，用于登录
`password` VARCHAR(20) NOT NULL,
reg_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP -- 注册时间默认当前时间
) ENGINE=InnoDB DEFAULT CHARSET utf8MB4;

-- 2. 商品表
CREATE TABLE ecommerce.product
(
product_id INT PRIMARY KEY,
pname VARCHAR(50) NOT NULL,
price DECIMAL(10, 2) NOT NULL,
stock INT NOT NULL DEFAULT 0,
CHECK (price >= 0), -- 单价非负
CHECK (stock >= 0), -- 库存非负
INDEX idx_product_name (pname) -- 普通索引（搜索商品）
) ENGINE=InnoDB DEFAULT CHARSET utf8MB4;

-- 3. 订单表
CREATE TABLE ecommerce.`order`
(
order_id INT PRIMARY KEY,
user_id INT NOT NULL,
order_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
total DECIMAL(10, 2) NOT NULL DEFAULT 0,
FOREIGN KEY (user_id) REFERENCES ecommerce.`user`(user_id),
CHECK (total >= 0), -- 总金额非负
INDEX idx_order_user (user_id) -- 普通索引（查用户的所有订单）
) ENGINE=InnoDB DEFAULT CHARSET utf8MB4;

-- 4. 订单项表
CREATE TABLE ecommerce.order_item
(
item_id INT PRIMARY KEY,
order_id INT NOT NULL,
product_id INT NOT NULL,
num INT NOT NULL,

```

```

price DECIMAL(10, 2) NOT NULL,
FOREIGN KEY (order_id) REFERENCES ecommerce.`order`(order_id),
FOREIGN KEY (product_id) REFERENCES ecommerce.product(product_id),
CHECK (num > 0), -- 购买数量大于等于1
CHECK (price >= 0), -- 单价非负
INDEX idx_item_order (order_id), -- 普通索引 (查订单详情)
INDEX idx_item_product (product_id) -- 普通索引 (查商品销售记录)
) ENGINE=InnoDB DEFAULT CHARSET utf8MB4;

-- 5. 地址表 (新增)
CREATE TABLE ecommerce.address
(
    addr_id INT PRIMARY KEY, -- 地址唯一标识
    user_id INT NOT NULL, -- 关联用户表
    receiver VARCHAR(20) NOT NULL, -- 收货人姓名
    phone VARCHAR(20) NOT NULL, -- 收货人电话
    detail VARCHAR(100) NOT NULL, -- 详细地址 (如街道门牌号)
    is_default TINYINT NOT NULL DEFAULT 0, -- 是否默认地址 (0=否, 1=是)
    -- 外键约束: 确保地址关联存在的用户
    FOREIGN KEY (user_id) REFERENCES ecommerce.`user`(user_id),
    -- 索引: 优化用户地址查询
    INDEX idx_addr_user (user_id)
) ENGINE=InnoDB DEFAULT CHARSET utf8MB4;

-- 6. 商品分类表 (新增)
CREATE TABLE ecommerce.category
(
    category_id INT PRIMARY KEY, -- 分类唯一标识
    category_name VARCHAR(30) NOT NULL UNIQUE, -- 分类名称 (如"电脑配件")
    parent_id INT DEFAULT NULL, -- 父分类ID (用于多级分类, 顶级分类为NULL)
    -- 自关联外键 (支持多级分类)
    FOREIGN KEY (parent_id) REFERENCES ecommerce.category(category_id)
    ON DELETE SET NULL -- 若父分类删除, 子分类的parent_id设为NULL
) ENGINE=InnoDB DEFAULT CHARSET utf8MB4;

```

部分结果如下 (user 表和 order_item 表):

Query 1

order_item - Table

order - Table

product - Table

user - Table

Table Name:

Schema: **ecommerce**

Charset/Collation:

utf8mb4

utf8mb4_0900_ai_ci

Engine:

InnoDB

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
user_id	INT	☒	☒	☒	☐	☐	☐	☐	☐	
name	VARCHAR(10)	☐	☒	☒	☐	☐	☐	☐	☐	
phone	VARCHAR(20)	☐	☒	☒	☐	☐	☐	☐	☐	
password	VARCHAR(20)	☐	☒	☒	☐	☐	☐	☐	☐	
reg_time	DATETIME	☐	☒	☒	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP

图 10: 用户 Columns 图

Query 1

user - Table

Table Name:

user

Schema:

ecommerce

Charset/Collation:

utf8mb4

utf8mb4_0900_ai_ci

Engine:

InnoDB

Comments:

Index Name

PRIMARY

Type

PRIMARY

phone

UNIQUE

Index Columns

Column

#

Order

Length

Index Options

Storage Type:

Key Block Size:

图 11: 用户 Indexs 图

Query 1

order_item - Table

Table Name:

order_item

Schema:

ecommerce

Charset/Collation:

utf8mb4

utf8mb4_0900_ai_ci

Engine:

InnoDB

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
item_id	INT	☒	☒	☐	☐	☐	☐	☐	☐	
order_id	INT	☐	☒	☐	☐	☐	☐	☐	☐	
product_id	INT	☐	☒	☐	☐	☐	☐	☐	☐	
num	INT	☐	☒	☐	☐	☐	☐	☐	☐	
price	DECIMAL(10,2)	☐	☒	☐	☐	☐	☐	☐	☐	

图 12: 订单项 Columns 图

Query 1

order_item - Table

Table Name:

order_item

Schema:

ecommerce

Charset/Collation:

utf8mb4

utf8mb4_0900_ai_ci

Engine:

InnoDB

Comments:

Index Name

Type

PRIMARY

PRIMARY

idx_item_order

INDEX

idx_item_product

INDEX

Index Columns

Column

#

Order

Length

Index Options

Storage Type:

Key Block Size:

Parser:

Visible:

☐

图 13: 订单项 Indexs 图

Query 1

order_item - Table

Table Name:

Schema: **ecommerce**

Charset/Collation:

utf8mb4

utf8mb4_0900_ai_ci

Engine:

InnoDB

Comments:

Foreign Key Name	Referenced Table	Column	Referenced Column	Foreign Key Options
order_item_ibfk_1	ecommerce : order			On Update:
order_item_ibfk_2	ecommerce : product			On Delete:

图 14: 订单项 Foreign Keys 图

6.2 插入虚拟数据

向 4 张表中插入 7-8 条虚拟数据，模拟电商系统初始状态：

```
-- 插入 7-8 条虚拟数据
-- 1. 用户表
INSERT INTO ecommerce.`user` (user_id, `name`, phone, `password`)
VALUES
(1, '张三', '13800138000', 'zhangsan123'),
(2, '李四', '13900139000', 'lisi456'),
(3, '王五', '13700137000', 'wangwu789'),
(4, '赵六', '13600136000', 'zhaoliu321'),
(5, '孙七', '13500135000', 'sunqi654'),
(6, '周八', '13400134000', 'zhouba987'),
(7, '吴九', '13300133000', 'wujiu765'),
(8, '郑十', '13200132000', 'zhengshi432');

-- 2. 商品表
INSERT INTO ecommerce.product (product_id, pname, price, stock)
VALUES
(101, '笔记本电脑', 4999.99, 50),
(102, '无线鼠标', 99.99, 200),
(103, '机械键盘', 199.99, 100),
(104, '蓝牙耳机', 299.99, 150),
(105, '移动硬盘', 599.99, 80),
(106, '显示器', 1299.99, 40),
(107, '充电宝', 129.99, 300),
(108, 'U盘', 69.99, 500);

-- 3. 订单表
INSERT INTO ecommerce.`order` (order_id, user_id, total)
VALUES
(1001, 1, 5199.98),      -- 张三：笔记本+键盘
(1002, 2, 99.99),        -- 李四：无线鼠标
(1003, 3, 1899.98),      -- 王五：显示器+U盘
(1004, 4, 299.99),       -- 赵六：蓝牙耳机
(1005, 5, 739.98),       -- 孙七：移动硬盘+U盘
(1006, 6, 129.99),       -- 周八：充电宝
(1007, 7, 6499.98);     -- 吴九：笔记本+显示器

-- 4. 订单项表
```

```

INSERT INTO ecommerce.order_item (item_id, order_id, product_id, num,
    price)
VALUES
(1, 1001, 101, 1, 4999.99),  -- 订单1001: 笔记本
(2, 1001, 103, 1, 199.99),  -- 订单1001: 键盘
(3, 1002, 102, 1, 99.99),   -- 订单1002: 鼠标
(4, 1003, 106, 1, 1299.99), -- 订单1003: 显示器
(5, 1003, 108, 1, 69.99),   -- 订单1003: U盘
(6, 1004, 104, 1, 299.99),  -- 订单1004: 耳机
(7, 1005, 105, 1, 599.99),  -- 订单1005: 移动硬盘
(8, 1005, 108, 2, 69.99);   -- 订单1005: 2个U盘

-- 5. 地址表
INSERT INTO ecommerce.address (addr_id, user_id, receiver, phone,
    detail, is_default)
VALUES
(1, 1, '张三', '13800138000', '北京市海淀区XX街道1号', 1),
(2, 2, '李四', '13900139000', '上海市浦东新区XX路2号', 1),
(3, 3, '王五', '13700137000', '广州市天河区XX巷3号', 0),
(4, 4, '赵六', '13600136000', '深圳市南山区XX街4号', 1),
(5, 5, '孙七', '13500135000', '杭州市西湖区XX路5号', 0),
(6, 6, '周八', '13400134000', '成都市武侯区XX巷6号', 1),
(7, 7, '吴九', '13300133000', '武汉市洪山区XX街7号', 0),
(8, 8, '郑十', '13200132000', '西安市雁塔区XX路8号', 1);

-- 6. 商品分类表
INSERT INTO ecommerce.category (category_id, category_name, parent_id
    )
VALUES
(1, '电脑设备', NULL),      -- 顶级分类
(2, '电脑配件', 1),        -- 子分类 (属于"电脑设备")
(3, '移动设备', NULL),     -- 顶级分类
(4, '存储设备', NULL),     -- 顶级分类
(5, '输入设备', 2),        -- 子分类 (属于"电脑配件")
(6, '音频设备', 3);        -- 子分类 (属于"移动设备")

-- 同步修改商品表, 增加分类关联字段 (仅新增1个字段, 不影响原有数据)
ALTER TABLE ecommerce.product
ADD COLUMN category_id INT,
ADD FOREIGN KEY (category_id) REFERENCES ecommerce.category(

```

```

category_id);

-- 为现有商品补充分类（保持原有商品数据不变）
UPDATE ecommerce.product SET category_id = 1 WHERE product_id = 101;
-- 笔记本电脑→电脑设备
UPDATE ecommerce.product SET category_id = 5 WHERE product_id = 102;
-- 无线鼠标→输入设备
UPDATE ecommerce.product SET category_id = 5 WHERE product_id = 103;
-- 机械键盘→输入设备
UPDATE ecommerce.product SET category_id = 6 WHERE product_id = 104;
-- 蓝牙耳机→音频设备
UPDATE ecommerce.product SET category_id = 4 WHERE product_id = 105;
-- 移动硬盘→存储设备
UPDATE ecommerce.product SET category_id = 2 WHERE product_id = 106;
-- 显示器→电脑配件
UPDATE ecommerce.product SET category_id = 3 WHERE product_id = 107;
-- 充电宝→移动设备
UPDATE ecommerce.product SET category_id = 4 WHERE product_id = 108;
-- U盘→存储设备
UPDATE ecommerce.product SET category_id = 6 WHERE product_id = 109;
-- 有线耳机→音频设备（对应之前新增的商品）

```

插入数据后的 4 张表如下：

Result Grid

Filter Rows:

Edit

	user_id	name	phone	password	reg_time
▶	1	张三	13800138000	zhangsan123	2025-10-14 22:45:43
	2	李四	13900139000	lisi456	2025-10-14 22:45:43
	3	王五	13700137000	wangwu789	2025-10-14 22:45:43
	4	赵六	13600136000	zhaoliu321	2025-10-14 22:45:43
	5	孙七	13500135000	sunqi654	2025-10-14 22:45:43
	6	周八	13400134000	zhouba987	2025-10-14 22:45:43
	7	吴九	13300133000	wujiu765	2025-10-14 22:45:43
	8	郑十	13200132000	zhengshi432	2025-10-14 22:45:43
✱	NULL	NULL	NULL	NULL	NULL

图 15: 用户表（插入数据后）

Result Grid		 Filter Rows:		Edit:		
	product_id	pname	price	stock		
▶	101	笔记本电脑	4999.99	50		
	102	无线鼠标	99.99	200		
	103	机械键盘	199.99	100		
	104	蓝牙耳机	299.99	150		
	105	移动硬盘	599.99	80		
	106	显示器	1299.99	40		
	107	充电宝	129.99	300		
	108	U盘	69.99	500		
*	NULL	NULL	NULL	NULL		

图 16: 商品表（插入数据后）

order_id	user_id	order_time	total
1001	1	2025-10-14 22:45:43	5199.98
1002	2	2025-10-14 22:45:43	99.99
1003	3	2025-10-14 22:45:43	1899.98
1004	4	2025-10-14 22:45:43	299.99
1005	5	2025-10-14 22:45:43	739.98
1006	6	2025-10-14 22:45:43	129.99
1007	7	2025-10-14 22:45:43	6499.98
NULL	NULL	NULL	NULL

图 17: 订单表（插入数据后）

item_id	order_id	product_id	num	price
1	1001	101	1	4999.99
2	1001	103	1	199.99
3	1002	102	1	99.99
4	1003	106	1	1299.99
5	1003	108	1	69.99
6	1004	104	1	299.99
7	1005	105	1	599.99
8	1005	108	2	69.99
NULL	NULL	NULL	NULL	NULL

图 18: 订单项表（插入数据后）

addr_id	user_id	receiver	phone	detail	is_default
1	1	张三	13800138000	北京市海淀区XX街道1号	1
2	2	李四	13900139000	上海市浦东新区XX路2号	1
3	3	王五	13700137000	广州市天河区XX巷3号	0
4	4	赵六	13600136000	深圳市南山区XX街4号	1
5	5	孙七	13500135000	杭州市西湖区XX路5号	0
6	6	周八	13400134000	成都市武侯区XX巷6号	1
7	7	吴九	13300133000	武汉市洪山区XX街7号	0
8	8	郑十	13200132000	西安市雁塔区XX路8号	1
NULL	NULL	NULL	NULL	NULL	NULL

图 19: 地址表（插入数据后）

category_id	category_name	parent_id
1	电脑设备	NULL
2	电脑配件	1
3	移动设备	NULL
4	存储设备	NULL
5	输入设备	2
6	音频设备	3
NULL	NULL	NULL

图 20: 商品分类表（插入数据后）

product_id	pname	price	stock	category_id
101	笔记本电脑	4999.99	50	1
102	无线鼠标	99.99	200	5
103	机械键盘	199.99	100	5
104	蓝牙耳机	299.99	150	6
105	移动硬盘	599.99	80	4
106	显示器	1299.99	40	2
107	充电宝	129.99	300	3
108	U盘	69.99	500	4
NULL	NULL	NULL	NULL	NULL

图 21: 商品表（更新 category_id 后）

6.3 数据库行为实际操作

6.3.1 数据查询

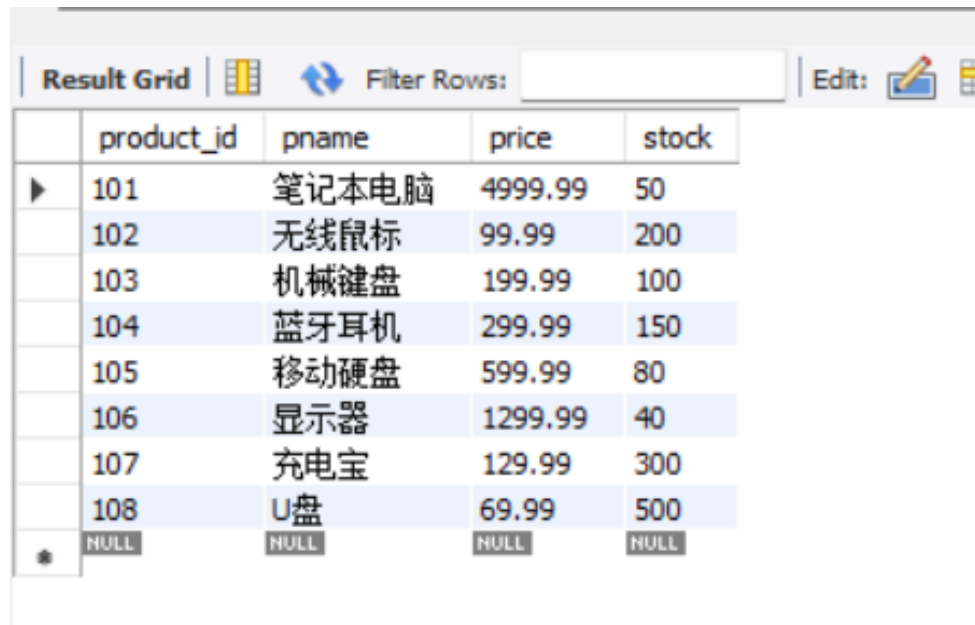
1. 单表查询示例

(1) 查询所有商品的信息

-- 查询所有商品的信息

```
SELECT * FROM ecommerce.product;
```

查询结果如下：



The screenshot shows a 'Result Grid' window with a toolbar at the top containing icons for 'Filter Rows' and 'Edit'. The grid displays the following data:

	product_id	pname	price	stock
▶	101	笔记本电脑	4999.99	50
	102	无线鼠标	99.99	200
	103	机械键盘	199.99	100
	104	蓝牙耳机	299.99	150
	105	移动硬盘	599.99	80
	106	显示器	1299.99	40
	107	充电宝	129.99	300
	108	U盘	69.99	500
●	NULL	NULL	NULL	NULL

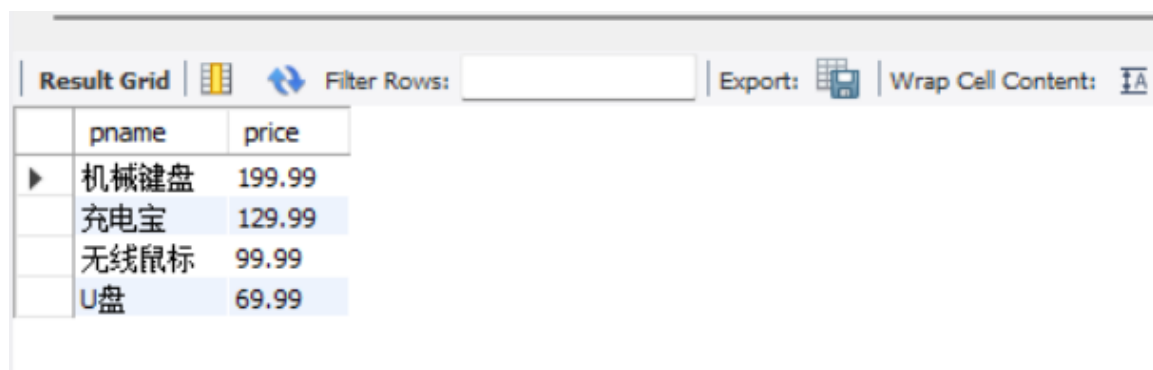
图 22: 所有商品信息

(2) 查询价格低于 200 元的商品名称和价格，按价格降序排列

-- 查询价格低于 200 元的商品名称和价格，按价格降序排列

```
SELECT pname, price  
FROM ecommerce.product  
WHERE price < 200  
ORDER BY price DESC;
```

查询结果如下：



The screenshot shows a 'Result Grid' window with a toolbar at the top containing icons for 'Filter Rows', 'Export', and 'Wrap Cell Content'. The grid displays the following data:

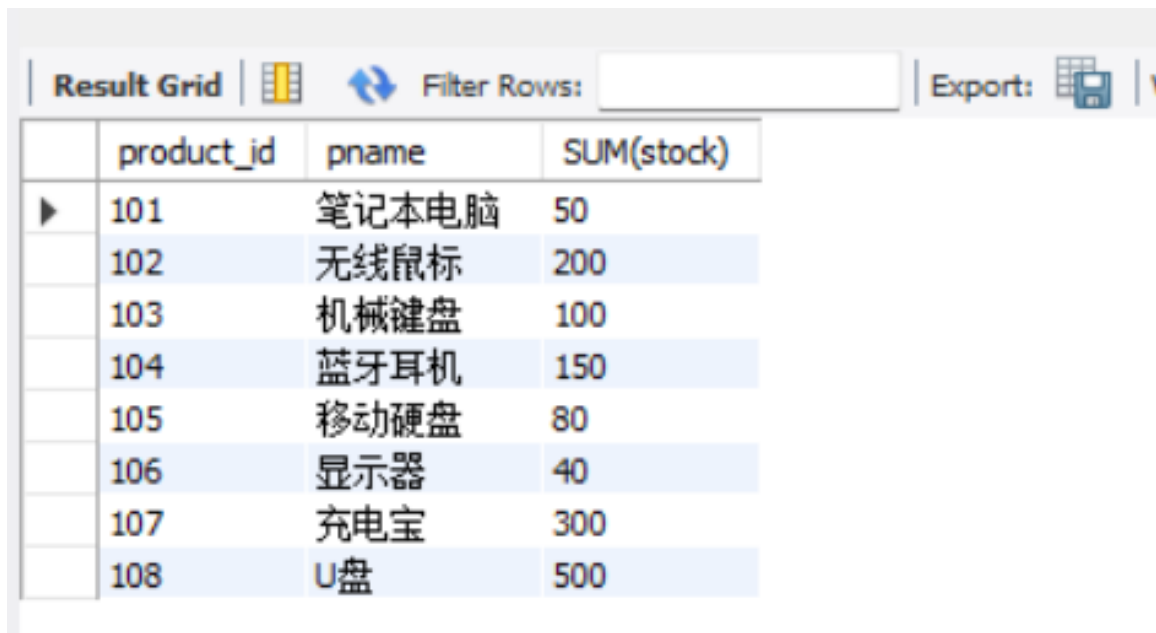
	pname	price
▶	机械键盘	199.99
	充电宝	129.99
	无线鼠标	99.99
	U盘	69.99

图 23: 查询价格低于 200 元的商品名称和价格，按价格降序排列

(3) 查询每个商品的库存总量

```
-- 查询每个商品的库存总量
SELECT product_id, pname, SUM(stock)
FROM ecommerce.product
GROUP BY product_id, pname;
```

查询结果如下：



The screenshot shows a database query result grid with the following data:

	product_id	pname	SUM(stock)
▶	101	笔记本电脑	50
	102	无线鼠标	200
	103	机械键盘	100
	104	蓝牙耳机	150
	105	移动硬盘	80
	106	显示器	40
	107	充电宝	300
	108	U盘	500

图 24: 查询每个商品的库存总量

2. 多表查询示例

(1) 查询订单信息和对应的用户名（等值查询）

```
-- 查询订单信息和对应的用户名（等值查询）
SELECT `order`.user_id, `order`.order_time, `order`.total, `user`.`
    name`
FROM ecommerce.`user`, ecommerce.`order`
WHERE `user`.user_id = `order`.user_id;
```

查询结果如下：

Result Grid				
Filter Rows:				
Export: Wrap Cell Content:				
	user_id	order_time	total	name
▶	1	2025-10-14 22:45:43	5199.98	张三
	2	2025-10-14 22:45:43	99.99	李四
	3	2025-10-14 22:45:43	1899.98	王五
	4	2025-10-14 22:45:43	299.99	赵六
	5	2025-10-14 22:45:43	739.98	孙七
	6	2025-10-14 22:45:43	129.99	周八
	7	2025-10-14 22:45:43	6499.98	吴九

图 25: 查询订单信息和对应的用户名

(2) 查询所有订单及其订单项（左外连接）

-- 查询所有订单及其订单项（左外连接）

```
SELECT `order`.user_id, order_item.product_id, order_item.num,
       order_item.price
FROM ecommerce.`order`
LEFT JOIN ecommerce.order_item ON `order`.order_id = order_item.
order_id;
```

查询结果如下：

Result Grid				
Filter Rows:				
Export: Wrap Cell Content:				
	user_id	product_id	num	price
▶	1	101	1	4999.99
	1	103	1	199.99
	2	102	1	99.99
	3	106	1	1299.99
	3	108	1	69.99
	4	104	1	299.99
	5	105	1	599.99
	5	108	2	69.99
	6	NULL	NULL	NULL
	7	NULL	NULL	NULL

图 26: 查询所有订单及其订单项

(3) 查询“电脑配件”分类下的所有商品（内连接）

-- 查询“电脑配件”分类下的所有商品

```
SELECT p.pname, p.price
FROM ecommerce.product
JOIN ecommerce.category ON product.category_id = category.category_id
WHERE category.category_name = '电脑配件';
```

查询结果如下：

Result Grid		
	pname	price
▶	显示器	1299.99

图 27: 查询“电脑配件”分类下的所有商品

3. 嵌套查询示例

查询购买过”蓝牙耳机”的用户名（子查询作为条件）

```
-- 查询购买过“蓝牙耳机”的用户名
SELECT `name`
FROM ecommerce.`user`
WHERE user_id IN
(
    SELECT user_id
    FROM ecommerce.`order`
    WHERE order_id IN
    (
        SELECT order_id
        FROM ecommerce.order_item
        WHERE product_id =
        (
            SELECT product_id
            FROM ecommerce.product
            WHERE pname = "蓝牙耳机"
        )
    )
);
```

查询结果如下：

Result Grid	Filter Rows:	Export:	Wrap Cell Content
name			
赵六			

图 28: 查询购买过”蓝牙耳机”的用户名

6.3.2 数据操作

1. 插入数据示例

-- 插入新商品

```
INSERT INTO ecommerce.product(product_id, pname, price, stock)
VALUES
(109, "有线耳机", 80.50, 250)
```

结果如下:

Result Grid



Filter Rows:

Edit:

	product_id	pname	price	stock
▶	101	笔记本电脑	4999.99	50
	102	无线鼠标	99.99	200
	103	机械键盘	199.99	100
	104	蓝牙耳机	299.99	150
	105	移动硬盘	599.99	80
	106	显示器	1299.99	40
	107	充电宝	129.99	300
	108	U盘	69.99	500
	109	有线耳机	80.50	250
✱	NULL	NULL	NULL	NULL

product 1 ✕

图 29: 插入新商品

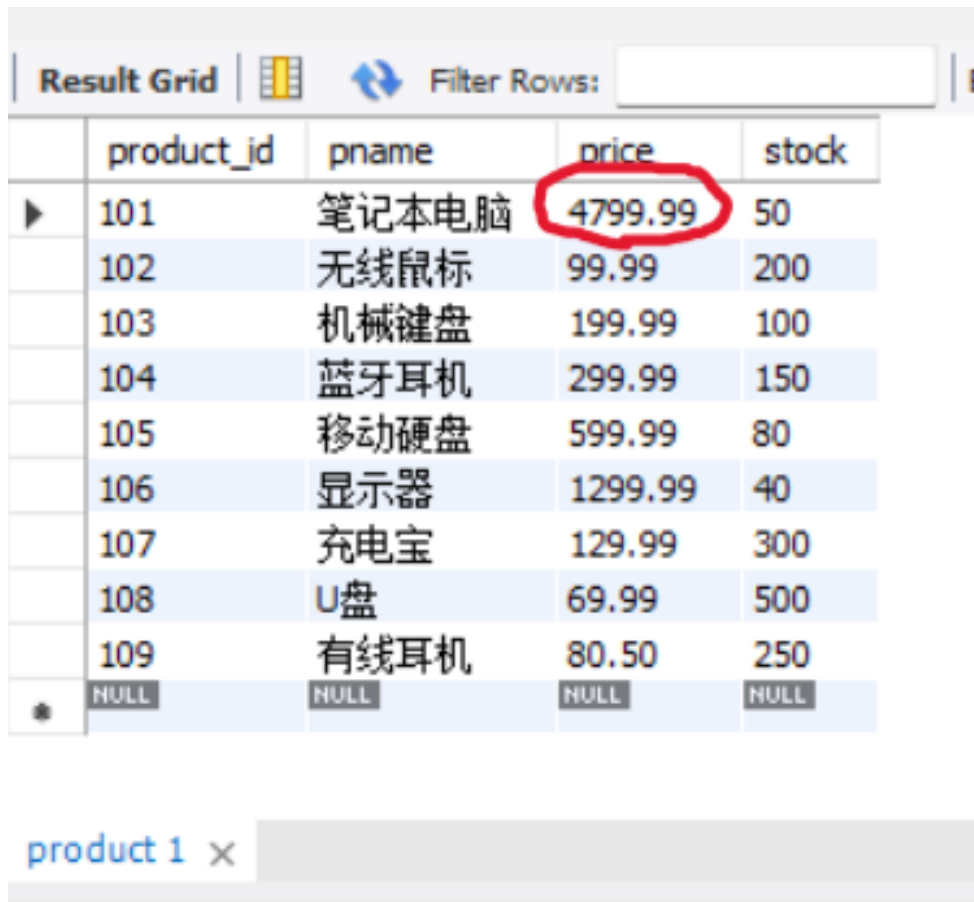
同理可插入新用户、订单、订单项，不多做演示。

2. 修改数据示例

(1) 修改商品价格 (笔记本电脑 4999.99 -> 4799.99)

```
-- 更新商品价格
UPDATE ecommerce.product
SET price = 4799.99
WHERE product_id = 101;
```

结果如下：



	product_id	pname	price	stock
▶	101	笔记本电脑	4799.99	50
	102	无线鼠标	99.99	200
	103	机械键盘	199.99	100
	104	蓝牙耳机	299.99	150
	105	移动硬盘	599.99	80
	106	显示器	1299.99	40
	107	充电宝	129.99	300
	108	U盘	69.99	500
	109	有线耳机	80.50	250
★	NULL	NULL	NULL	NULL

product 1 x

图 30: 更新商品价格

(2) 下单后减少商品库存

```
-- 下单后减少商品库存
UPDATE ecommerce.product
INNER JOIN ecommerce.order_item ON product.product_id = order_item.
    product_id
SET product.stock = product.stock - order_item.num -- 用订单项中的实
    际购买数量减少库存
WHERE order_item.order_id = 1005;
```

结果如下：

Result Grid   Filter Rows:				
	product_id	pname	price	stock
▶	101	笔记本电脑	4799.99	50
	102	无线鼠标	99.99	200
	103	机械键盘	199.99	100
	104	蓝牙耳机	299.99	150
	105	移动硬盘	599.99	80
	106	显示器	1299.99	40
	107	充电宝	129.99	300
	108	U盘	69.99	498
	109	有线耳机	80.50	250
★	NULL	NULL	NULL	NULL

product 1 ×

图 31: 下单后减少商品库存

3. 删除数据示例

- 取消订单同时删除订单项
- 必须先删除订单对应的订单项（外键依赖），否则报错

DELETE

FROM ecommerce.order_item

WHERE order_id = 1003;

DELETE

FROM ecommerce.`order`

WHERE order_id = 1003;

结果如下：

order_id	user_id	order_time	total
1001	1	2025-10-14 22:45:43	5199.98
1002	2	2025-10-14 22:45:43	99.99
1004	4	2025-10-14 22:45:43	299.99
1005	5	2025-10-14 22:45:43	739.98
1006	6	2025-10-14 22:45:43	129.99
1007	7	2025-10-14 22:45:43	6499.98
NULL	NULL	NULL	NULL

图 32: 订单表

item_id	order_id	product_id	num	price
1	1001	101	1	4999.99
2	1001	103	1	199.99
3	1002	102	1	99.99
6	1004	104	1	299.99
7	1005	105	1	599.99
8	1005	108	2	69.99
NULL	NULL	NULL	NULL	NULL

图 33: 订单项表

6.3.3 视图操作

1. 用户订单概览视图

-- 创建用户订单概览视图

```
USE ecommerce;
```

```
CREATE VIEW user_order AS
```

```
SELECT `user`.user_id, `user`.`name`, `user`.`phone`, `order`.  
order_id, `order`.order_time, `order`.total
```

```
FROM ecommerce.`user`
```

```
LEFT JOIN ecommerce.`order` ON `user`.user_id = `order`.user_id;
```

user_id	name	phone	order_id	order_time	total
1	张三	13800138000	1001	2025-10-14 22:45:43	5199.98
2	李四	13900139000	1002	2025-10-14 22:45:43	99.99
3	王五	13700137000	NULL	NULL	NULL
4	赵六	13600136000	1004	2025-10-14 22:45:43	299.99
5	孙七	13500135000	1005	2025-10-14 22:45:43	739.98
6	周八	13400134000	1006	2025-10-14 22:45:43	129.99
7	吴九	13300133000	1007	2025-10-14 22:45:43	6499.98
8	郑十	13200132000	NULL	NULL	NULL

图 34: 用户订单概览视图

2. 商品销售统计视图

-- 商品销售统计视图

```
USE ecommerce;
```

```

CREATE VIEW product_order_item AS
SELECT product.product_id, product.pname, product.price AS price1,
       product.stock, order_item.num, order_item.price AS price2
FROM product
JOIN order_item ON product.product_id = order_item.product_id;

```

	product_id	pname	price	stock
▶	101	笔记本电脑	4999.99	50
	102	无线鼠标	99.99	200
	103	机械键盘	199.99	100
	104	蓝牙耳机	299.99	150
	105	移动硬盘	599.99	80
	106	显示器	1299.99	40
	107	充电宝	129.99	300
	108	U盘	69.99	500
	109	有线耳机	80.50	250
★	NULL	NULL	NULL	NULL

product 1 x

图 35: 商品销售统计试图

3. 订单商品明细视图

```

-- 订单商品明细视图
USE ecommerce;

CREATE VIEW product_details AS
SELECT `order`.order_id, `order`.order_time, product.price AS price1,
       product.pname, order_item.num, order_item.price AS price2
FROM `order`
JOIN order_item ON `order`.order_id = order_item.order_id
JOIN product ON order_item.product_id = product.product_id;

```

Result Grid  Filter Rows: Edit: [				
	product_id	pname	price	stock
▶	101	笔记本电脑	4999.99	50
	102	无线鼠标	99.99	200
	103	机械键盘	199.99	100
	104	蓝牙耳机	299.99	150
	105	移动硬盘	599.99	80
	106	显示器	1299.99	40
	107	充电宝	129.99	300
	108	U盘	69.99	500
	109	有线耳机	80.50	250
★	NULL	NULL	NULL	NULL

product 1 x

图 36: 订单商品明细视图

4. 用户地址关联视图

```
-- 用户地址关联视图
USE ecommerce;

CREATE VIEW user_addr AS
SELECT `user`.user_id, `user`.`name`, address.receiver, address.phone
      AS receiver_address, address.detail AS detail_address
FROM ecommerce.`user`
LEFT JOIN ecommerce.address ON `user`.user_id = address.user_id AND
      address.is_default=1; -- 仅关联默认地址
```

Result Grid Filter Rows: Export: Wrap Cell Co					
	user_id	name	receiver	receiver_address	detail_address
▶	1	张三	张三	13800138000	北京市海淀区XX街道1号
	2	李四	李四	13900139000	上海市浦东新区XX路2号
	3	王五	NULL	NULL	NULL
	4	赵六	赵六	13600136000	深圳市南山区XX街4号
	5	孙七	NULL	NULL	NULL
	6	周八	周八	13400134000	成都市武侯区XX巷6号
	7	吴九	NULL	NULL	NULL
	8	郑十	郑十	13200132000	西安市雁塔区XX路8号

图 37: 用户地址关联视图

5. 商品分类关联视图

-- 订单商品明细视图

-- 商品分类关联视图

```
USE ecommerce;
```

```
CREATE VIEW product_category AS
```

```
SELECT product.product_id, product.pname, product.price, category.
       category_name, category.parent_id
```

```
FROM ecommerce.product
```

```
LEFT JOIN ecommerce.category ON product.category_id = category.
       category_id;
```

	product_id	pname	price	category_name	parent_id
▶	101	笔记本电脑	4999.99	电脑设备	NULL
	102	无线鼠标	99.99	输入设备	2
	103	机械键盘	199.99	输入设备	2
	104	蓝牙耳机	299.99	音频设备	3
	105	移动硬盘	599.99	存储设备	NULL
	106	显示器	1299.99	电脑配件	1
	107	充电宝	129.99	移动设备	NULL
	108	U盘	69.99	存储设备	NULL

图 38: 商品分类关联视图